Clase 17 · Servidor concurrente en vivo y select()

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 22 oct 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Prueba en vivo del Programa 06 (servidor concurrente con fork()), detalles del código, bitácora con struct tm, puertos efímeros y el salto a select() para un chat grupal (Programa 07). Continúa la Clase 16.

1. Cliente contra servidor en el código

> **Pregunta de examen teórico**
>
> El **servidor** declara **dos** `struct sockaddr_in` (la propia, con IP y puerto de escucha, y otra para el cliente aceptado). El **cliente** declara **una** (IP y puerto del servidor al que se conecta).

`LINE_MAX` define el tamaño de los buffers de texto leído por línea (hasta el Enter) en vez de fijar un número a mano. Solo sirve para texto terminado en salto de línea, no para datos binarios.

2. Buffers dentro del hijo y copia de la IP

- En el Programa 06, `buff_peticion` y `buff_respuesta` se declaran **dentro de `if (pid == 0)`**: el padre, que solo acepta conexiones, no reserva ni duplica en cada `fork()` memoria que no usa.
- La IP que devuelve `gethostbyname` (estructura `hostent`) se copia a `sin_addr` con `memcpy`, usando `h_length` (4 bytes en IPv4). Lo usual es copiar `h_addr_list[0]` (o `h_addr`); revisar el código del profesor.

3. Bitácora de fecha, hora e IP con struct tm

Justo después de `accept()` se registra el instante y la IP del cliente:

1. `time_t t; time(&t);` → segundos desde la época Unix.
2. `localtime(&t)` → puntero a `struct tm`.
3. Campos: `tm_mday` (día), **`tm_mon + 1`** (mes, cuenta desde 0), **`tm_year + 1900`** (año, cuenta desde 1900), `tm_hour`, `tm_min`, `tm_sec`.
4. Imprimir o guardar "día/mes/año hh:mm:ss, cliente conectado desde [IP]".

Campo crudo (oct 2026)	Corregido
`tm_mon` = 9	9 + 1 = **10**
`tm_year` = 126	126 + 1900 = **2026**

Es la lógica con la que el profesor auditó en su servidor 132.248.59.6 las entregas del examen parcial remoto (IP, fecha y hora de cada alumno).

4. Cierre cruzado de descriptores tras fork()

`accept()` devuelve un descriptor nuevo para el cliente (`fd_c`) y deja el de escucha (`fd_s`). `fork()` duplica la tabla de descriptores: padre e hijo tienen **ambos** abiertos.

Proceso	Cierra	Conserva y hace
Hijo (`pid == 0`)	`close(fd_s)`: no aceptará conexiones	Usa `fd_c` con `recv`/`send`; al recibir "adiós" cierra `fd_c` y `exit(0)`
Padre (`else`)	`close(fd_c)`: no conversa con el cliente	Conserva `fd_s` y regresa a `accept()` en el `while(1)`

Resuelve la limitación del servidor secuencial (Programa 05). Error común: olvidar alguno de los dos `close()` y agotar el límite de descriptores del sistema.

5. Demostración y puertos efímeros

- Servidor en 132.248.59.6: `./programa_06_servidor_inet 4898`. Se conectaron alumnos desde terminales, navegadores y un teléfono con datos móviles.
- Con `ss -atnp` se vio a varios alumnos de la facultad con la **misma IP pública** (132.247.100.226) y **puertos de origen distintos** (49802, 48469, 49808).

> **Puerto efímero**
>
> El sistema operativo del cliente asigna a cada conexión un puerto de origen temporal. Cada conexión es la tupla (IP origen, puerto efímero, IP destino, puerto destino), así que conexiones desde la misma IP no chocan.
>
> - Los navegadores imprimieron en el servidor sus cabeceras HTTP (User-Agent de Android, que usa kernel Linux).
> - Retoma la clasificación de puertos (sistema, usuario, dinámicos) de la Clase 9. El rango exacto de efímeros no es cifra para memorizar.

6. De fork() a select(): chat grupal (Programa 07)

Problema: cada hijo de `fork()` tiene su propio espacio de memoria. Si el cliente 1 escribe "hola a todos", el hijo 1 lo recibe pero no puede pasarlo a los hijos 2 y 3 (haría falta tuberías o memoria compartida).

Solución: un **solo proceso** servidor, sin `fork()`, que vigila varios descriptores a la vez con `select()`. El tipo `fd_set` es una "bolsa" de descriptores:

Macro	Qué hace
`FD_ZERO(&readfds)`	Vacía el conjunto
`FD_SET(fd, &readfds)`	Agrega un descriptor (servidor o cliente)
`FD_CLR(fd, &readfds)`	Lo quita (cliente que se va)

`FD_ISSET(fd, &readfds)`	¿Ese descriptor tiene lectura lista?

Lógica de `programa_07_servidor_select.c` (reconstrucción de NotebookLM):

1. Arreglo `client_socket[max_clients]` en ceros.
2. En cada ciclo: `FD_ZERO`, `FD_SET` del servidor y de cada cliente, actualizar `max_fd`.
3. `select(max_fd + 1, &readfds, NULL, NULL, NULL)` se bloquea hasta que haya actividad.
4. Si es el socket servidor → conexión nueva: `accept()` y guardar el socket en el arreglo.
5. Si es un cliente → `recv()` y reenviar con `send()` a todos los demás.

Errores comunes: no llamar `FD_ZERO` antes de rellenar el conjunto en cada ciclo; no actualizar `max_fd`.

Avisos

- Sin entrega obligatoria. Tarea moral: probar en local el Programa 06 (servidor y cliente, varias terminales) y revisar el Programa 07.
- Próxima clase: prueba en vivo del chat con `select()`, con más de 10 alumnos conectados a la vez.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Reconocer el código. Servidor: dos `sockaddr_in`; cliente: una.

> **fork() y descriptores (examen)**
>
> Tras `fork()` ambos procesos tienen `fd_s` y `fd_c`. Hijo: `close(fd_s)` y atiende con `fd_c`. Padre: `close(fd_c)` y vuelve a `accept()` con `fd_s`.

Bitácora. `time()` → `localtime()` → `struct tm`; mes = `tm_mon + 1`, año = `tm_year + 1900` (9 → 10, 126 → 2026).

Puertos efímeros. Misma IP pública, distinto puerto de origen asignado por el SO: cada conexión es una tupla única.

Modelo	Para recordar
Concurrente con `fork()`	Un proceso por cliente; memoria aislada: no sirve para chat grupal
Multiplexado con `select()`	Un solo proceso vigila muchos descriptores; reenvía mensajes a todos
`fd_set`	`FD_ZERO`, `FD_SET`, `FD_CLR`, `FD_ISSET`; `select(max_fd + 1, …)`